Learning Objectives

Learners will be able to…

- **Describe the structure and functionality of binary heaps and how they can effectively implement priority queues.**
- **Construct priority queues using heap data structures to showcase their efficiency and application.**
- **Analyze and articulate the time and space complexity associated with the key operations (insertion, deletion, and peeking) in heap-based priority queues, emphasizing their impact on computational efficiency.**
- **Implement and demonstrate the functionality of Double-Ended Priority Queues (DEPQs), showcasing the ability to efficiently manage both maximum and minimum priority elements within the same data structure.**

Priority Queues and Binary Heaps

Priority queues are abstract data types that operate similarly to regular queues but with an added feature: each element has a priority associated with it. In a priority queue, elements are served based on their priority rather than just their order in the queue.

- **High-Priority Processing**: Elements with higher priority are processed before those with lower priority.
- **Real-World Examples**: Priority queues are used in scenarios like managing tasks in an operating system, where certain processes need to be executed before others.

Binary Heaps as Priority Queues

Binary heaps, either max-heaps or min-heaps, are commonly used to implement priority queues due to their efficiency in managing the hierarchical structure of priorities.

- **Max-Heap for Highest Priority**: In a max-heap, the highest priority element (the maximum value) is always at the root.
- **Min-Heap for Lowest Priority**: In a min-heap, the lowest priority element (the minimum value) is at the root.

Example of a Priority Queue using a Max-Heap:



Operations in Binary Heap Priority Queues

1. **Insertion**: Adding a new element to the heap while maintaining the heap structure and properties.
   - The element is initially placed at the end of the heap and then “heapified up” to its correct position.
2. **Extraction**: Removing the element with the highest (or lowest) priority.
   - In a max-heap, this is the root. The last element is moved to the root and then “heapified down.”

Efficiency of Binary Heaps in Priority Queues

Binary heaps provide efficient implementations for the operations of priority queues:

- **Insertion**: O(log n) due to the heapify up process.
- **Extraction**: O(log n) as it involves removing the root and heapifying down.

Use Cases of Binary Heap Priority Queues

Binary heap-based priority queues are used in various applications, including:

- **Task Scheduling**: Managing tasks based on priority levels.
- **Dijkstra's Algorithm**: Selecting the next node with the shortest distance in graph algorithms.
- **Huffman Coding**: Building the Huffman tree for data compression.

Binary heaps offer a balance of simplicity and efficiency, making them a popular choice for implementing priority queues in numerous computational scenarios. Their ability to quickly access and manage the highest or lowest priority elements makes them indispensable in algorithms and systems where priority management is crucial.

Representing Priority Queues as Heaps

Priority queues are abstract data types that prioritize elements based on certain criteria. In a priority queue, elements with higher priority are served before those with lower priority. Heaps, particularly binary heaps, are an efficient way to implement priority queues due to their ability to maintain order based on priority with optimal time complexity.

Structure

A priority queue implemented using a heap typically uses a max-heap for a queue where higher values indicate higher priority. Conversely, a min-heap is used when lower values signify higher priority.

Example Structure (Max-Heap):

```
      50
     /  \
   30    20
  / \   / \
 10 15 5   7
```

In this max-heap, `50` has the highest priority and would be the first to be dequeued.

Example Structure (Min-Heap):

```
      5
     / \
   7    10
  / \   / \
 15 30 50 20
```

In this min-heap, `5` has the highest priority and would be the first to be dequeued.

Operations

Insertion

Inserting an element into a priority queue implemented as a heap follows the standard heap insertion process, ensuring the heap property is maintained.

Initial Heap:

```
        50
       /  \
     30    20
    / \   / \
   10 15 5   7
```

Example: Insert `25`

1. **Initial Placement**

   Place `25` at the next available position, which is the leftmost position at the bottom level.

   ```
          50
         /  \
       30    20
      / \   / \
     10 15 5   7
    /
   25
   ```

2. **Heapify Up**

   Compare `25` with its parent (`10`) and swap if necessary. Repeat until the heap property is restored.

   Since `25` is greater than `10`, they are swapped.

   ```
          50
         /  \
       30    20
      / \   / \
     25 15 5   7
    /
   10
   ```

   Continue comparing `25` with its new parent (`30`). No swap is needed as `30` is greater than `25`. This makes the heap above the final heap after insertion.

Deletion (Extract Max/Min)

The process of removing the element with the highest priority (the root of the heap) involves replacing the root with the last element and then “heapifying down.”

Initial Heap:

```
        50
       /  \
     30    20
    / \   / \
  25 15 5   7
 /
10
```

Example: Remove `50`

1. **Remove Root and Replace with Last Element**

   Replace the root (`50`) with the last element (`10`).

   ```
         10
        /  \
      30    20
     / \   / \
   25 15 5   7
   ```

2. **Heapify Down**

   Compare `10` with its children (`30` and `20`) and swap with the larger one (`30`).

   ```
         30
        /  \
      10    20
     / \   / \
   25 15 5   7
   ```

   Continue heapifying down. Now compare `10` with its children (`25` and `15`) and swap with the larger one (`25`).

   ```
      30
     /  \
   25   20
  / \   / \
 10 15 5   7
   ```

   The heapifying process ends here as `10` has no children, resulting in the final heap as shown above. It's important to note the significance of the heapifying process in ensuring that after the deletion of the root (`50`), the heap property is maintained by appropriately rearranging the elements.

Conclusion

Implementing priority queues with heaps provides an efficient and effective way to manage data based on priority. The binary heap structure, whether max-heap or min-heap, ensures optimal performance for both insertion and deletion, making it a popular choice for various applications.

Implementing Priority Queues with Heaps

In this lesson, we'll implement a priority queue using a heap. We'll focus on using a max-heap, where higher values indicate higher priority.

Note that we will be using the `HeapImplementation` class from previously as a basis for implementing priority queues as heaps. The `HeapImplementation` class is already provided for you.

Priority Queue Class

First, we'll define a `PriorityQueueImplementation` class that uses the `HeapImplementation` structure.

```
class PriorityQueueImplementation {
private:
    HeapImplementation heap;

public:
    PriorityQueueImplementation(int capacity) : heap(capacity)
        {}

    void enqueue(int item) {
        heap.insert(item);
    }

    int dequeue() {
        return heap.deleteRoot();
    }

    void printQueue() {
        heap.printHeap();
    }
};
```

Add this `PriorityQueueImplementation` class to your student file on the left. Place this class into the specified space in the code file.

Enqueue Operation

The `enqueue` method adds an element to the `priority` queue.

```
void enqueue(int item) {
    heap.insert(item);
}
```

Dequeue Operation

The `dequeue` method removes and returns the element with the highest priority (the root of the max-heap).

```
int dequeue() {
    return heap.deleteRoot();
}
```

Testing the Priority Queue

To test our priority queue, we'll want to add a `main` method.

```
int main() {
    PriorityQueueImplementation pq(10);
    pq.enqueue(30);
    pq.enqueue(20);
    pq.enqueue(50);
    pq.enqueue(15);
    pq.enqueue(10);
    pq.enqueue(5);
    pq.enqueue(7);

    std::cout << "Priority Queue after enqueues:" << std::endl;
    pq.printQueue();

    std::cout << "Dequeued item: " << pq.dequeue() << std::endl;

    std::cout << "Priority Queue after dequeue:" << std::endl;
    pq.printQueue();

    return 0;
}
```

Add this `main` method to your student file underneath the `printQueue` method within the `PriorityQueueImplementation` class.

Student's Code File

Here's the complete code for your `PriorityQueueImplementation.cpp` file:

```
class HeapImplementation {
private:
```

```
    int* heap;
    int size;
    int capacity;

    void resize() {
        capacity *= 2;
        int* newHeap = new int[capacity];
        for (int i = 0; i < size; ++i) {
            newHeap[i] = heap[i];
        }
        delete[] heap;
        heap = newHeap;
    }

    void heapifyUp(int index) {
        while (hasParent(index) && parent(index) < heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

    void heapifyDown() {
        int index = 0;
        while (hasLeftChild(index)) {
            int largerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) >
        leftChild(index)) {
                largerChildIndex = getRightChildIndex(index);
            }
            if (heap[index] < heap[largerChildIndex]) {
                swap(index, largerChildIndex);
            } else {
                break;
            }
            index = largerChildIndex;
        }
    }

    bool hasParent(int i) { return getParentIndex(i) >= 0; }
    int parent(int i) { return heap[getParentIndex(i)]; }
    int getParentIndex(int i) { return (i - 1) / 2; }
    bool hasLeftChild(int i) { return getLeftChildIndex(i) <
        size; }
    bool hasRightChild(int i) { return getRightChildIndex(i) <
        size; }
    int leftChild(int i) { return heap[getLeftChildIndex(i)]; }
    int rightChild(int i) { return heap[getRightChildIndex(i)];
        }
    int getLeftChildIndex(int i) { return 2 * i + 1; }
    int getRightChildIndex(int i) { return 2 * i + 2; }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

public:
```

```
    HeapImplementation(int capacity) : size(0),
        capacity(capacity) {
        heap = new int[capacity];
    }

    void insert(int value) {
        if (size == capacity) {
            resize();
        }
        heap[size] = value;
        heapifyUp(size);
        size++;
    }

    int deleteRoot() {
        if (size == 0) {
            throw std::out_of_range("Heap is empty");
        }
        int root = heap[0];
        heap[0] = heap[size - 1];
        size--;
        heapifyDown();
        return root;
    }

    void printHeap() {
        for (int i = 0; i < size; ++i) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
};

class PriorityQueueImplementation {
private:
    HeapImplementation heap;

public:
    PriorityQueueImplementation(int capacity) : heap(capacity)
        {}

    void enqueue(int item) {
        heap.insert(item);
    }

    int dequeue() {
        return heap.deleteRoot();
    }

    void printQueue() {
        heap.printHeap();
    }
};

int main() {
    PriorityQueueImplementation pq(10);
    pq.enqueue(30);
    pq.enqueue(20);
```

```
    pq.enqueue(50);
    pq.enqueue(15);
    pq.enqueue(10);
    pq.enqueue(5);
    pq.enqueue(7);

    std::cout << "Priority Queue after enqueues:" << std::endl;
    pq.printQueue();

    std::cout << "Dequeued item: " << pq.dequeue() << std::endl;

    std::cout << "Priority Queue after dequeue:" << std::endl;
    pq.printQueue();

    return 0;
}
```

This code includes the `PriorityQueueImplementation` class with `enqueue` and `dequeue` methods, and a `main` method for testing.

Time and Space Complexity in Heap-Based Priority Queues

In computer science, understanding the time and space complexity of algorithms and data structures is crucial for evaluating their efficiency. When it comes to heap-based priority queues, both these complexities play a significant role in determining the performance, especially in large-scale applications.

Heap Structure Overview

A heap, particularly a binary heap, is a complete binary tree structure. It's used to implement priority queues efficiently due to its ability to maintain the heap property (either max-heap or min-heap) in a balanced manner.

Time Complexity

The time complexity of an algorithm indicates the amount of time it takes to run, often expressed in terms of the size of the input (n).

1. **Insertion (enqueue)**:
    - **Process**: Inserting a new element into a heap involves adding the element at the end of the heap array and then performing "heapify up" operations to maintain the heap property.
    - **Complexity**: O(log n), because in the worst case, a newly inserted element might need to be swapped until it reaches the root.
2. **Deletion (dequeue)**:
    - **Process**: Deletion in a heap-based priority queue typically means removing the root element. This is followed by moving the last element to the root and performing "heapify down" operations.
    - **Complexity**: O(log n), as the element may need to be swapped down through the heap's levels to maintain the heap property.
3. **Peek**:
    - **Process**: Accessing the root element (peek) to view the highest (or lowest) priority element.
    - **Complexity**: O(1), as it involves no traversal and directly accesses the root.
4. **Heapify Operations**:
    - **Process**: Both "heapify up" and "heapify down" operations are

essential for maintaining the heap structure during insertions and deletions.
    - **Complexity**: O(log n), since these operations may involve traversing the height of the heap.

Space Complexity

Space complexity measures the amount of memory an algorithm needs in relation to the input size.

- **Heap Storage**:
    - **Process**: A heap is a complete binary tree and can be efficiently represented using an array. Each element in the heap requires space in this array.
    - **Complexity**: O(n), where n is the number of elements in the priority queue. This linear space complexity is due to the need to store each element in the heap structure.

Factors Influencing Complexity

- **Heap Type**: The complexity remains the same for both max-heaps and min-heaps.
- **Heap Shape**: Since a binary heap is a complete binary tree, it ensures that the tree remains balanced, directly influencing the logarithmic nature of its time complexity.
- **Data Distribution**: The distribution of the elements can affect the number of operations required for heapification, although the worst-case complexity remains the same.

Conclusion

Understanding the time and space complexities of heap-based priority queues is essential for evaluating their suitability in different applications. Their logarithmic time complexity for insertion and deletion operations and linear space complexity make them highly efficient for a variety of use cases, especially where priority management is key.

Remember, the efficiency of a data structure is not just about its theoretical complexity but also how it fits into the broader context of the application's requirements and constraints.

Double-Ended Priority Queues (DEPQ)

Double-Ended Priority Queues (DEPQs) are an advanced data structure that allows efficient access and operations at both ends of the priority spectrum. They are particularly useful in scenarios where it is crucial to access both the smallest and largest elements quickly.

Characteristics of DEPQ

Dual Access: Enables finding and removing both the minimum and maximum elements efficiently.
Hybrid Structure: Often implemented using a combination of a max-heap and a min-heap.

Implementation in C++

We will continue to use our `HeapImplementation` class for this lesson which is already provided to you to the left. However, note that a new method `remove` has been added in order to remove a specific element from the heap. This is necessary since the `deleteRoot` method only removes the root element.

```
// Base HeapImplementation class
class HeapImplementation {
protected:
    int* heap;
    int size;
    int capacity;

public:
    HeapImplementation(int capacity) : size(0),
        capacity(capacity) {
        heap = new int[capacity];
    }

protected:
    void resize() {
        capacity *= 2;
        int* resizedHeap = new int[capacity];
        for (int i = 0; i < size; i++) {
            resizedHeap[i] = heap[i];
        }
        delete[] heap;
        heap = resizedHeap;
    }

public:
    void insert(int value) {
```

```
        if (size == capacity) {
            resize();
        }
        heap[size] = value;
        heapifyUp();
        size++;
    }

    virtual void heapifyUp() {
        int index = size;
        while (hasParent(index) && parent(index) < heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

public:
    int deleteRoot() {
        if (size == 0) {
            throw std::out_of_range("Heap is empty");
        }
        int root = heap[0];
        heap[0] = heap[size - 1];
        size--;
        heapifyDown();
        return root;
    }

    virtual void heapifyDown() {
        int index = 0;
        while (hasLeftChild(index)) {
            int largerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) >
        leftChild(index)) {
                largerChildIndex = getRightChildIndex(index);
            }
            if (heap[index] < heap[largerChildIndex]) {
                swap(index, largerChildIndex);
            } else {
                break;
            }
            index = largerChildIndex;
        }
    }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    bool hasParent(int i) { return getParentIndex(i) >= 0; }
    int parent(int i) { return heap[getParentIndex(i)]; }
    int getParentIndex(int i) { return (i - 1) / 2; }
    bool hasLeftChild(int i) { return getLeftChildIndex(i) <
        size; }
```

```
    bool hasRightChild(int i) { return getRightChildIndex(i) <
        size; }
    int leftChild(int i) { return heap[getLeftChildIndex(i)]; }
    int rightChild(int i) { return heap[getRightChildIndex(i)];
        }
    int getLeftChildIndex(int i) { return 2 * i + 1; }
    int getRightChildIndex(int i) { return 2 * i + 2; }

public:
    bool remove(int element) {
        for (int i = 0; i < size; i++) {
            if (heap[i] == element) {
                swap(i, size - 1);
                size--;
                heapifyDown();
                return true;
            }
        }
        return false;
    }

    void printHeap() {
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
};
```

MaxHeap and MinHeap Class Structures

We will add two new classes called `MaxHeap` and `MinHeap`. These will enable us to make use of both a max-heap and a min-heap simultaneously. The `MaxHeap` class will simply inherit from the `HeapImplementation` class since `HeapImplementation` was developed with a max-heap in mind. However, in the `MinHeap` class, we will need to override both the `heapifyUp` and `heapifyDown` methods. These are revised to work specifically with a min-heap.

```
// MaxHeap class
class MaxHeap : public HeapImplementation {
public:
    MaxHeap(int capacity) : HeapImplementation(capacity) {}
};
```

```
// MinHeap class
class MinHeap : public HeapImplementation {
public:
    MinHeap(int capacity) : HeapImplementation(capacity) {}

    void heapifyUp() override {
        int index = size;
        while (hasParent(index) && parent(index) > heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

    void heapifyDown() override {
        int index = 0;
        while (hasLeftChild(index)) {
            int smallerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) <
        leftChild(index)) {
                smallerChildIndex = getRightChildIndex(index);
            }
            if (heap[index] > heap[smallerChildIndex]) {
                swap(index, smallerChildIndex);
            } else {
                break;
            }
            index = smallerChildIndex;
        }
    }
};
```

The DEPQ Class Structure

Next, we will work on the Double-Ended Priority Queue (DEPQ) implementation. Here, we will incorporate both the `MaxHeap` and `MinHeap` classes. The add method will insert elements into both heaps. The `removeMax` and `removeMin` methods remove an element from one heap and then also remove that corresponding element from the other heap.

```
// DoubleEndedPriorityQueue class
class DoubleEndedPriorityQueue {
private:
    MaxHeap maxHeap;
    MinHeap minHeap;

public:
    DoubleEndedPriorityQueue(int capacity) : maxHeap(capacity),
        minHeap(capacity) {}

    void add(int element) {
        maxHeap.insert(element);
        minHeap.insert(element);
    }

    int removeMax() {
        int max = maxHeap.deleteRoot();
        minHeap.remove(max);
        return max;
    }

    int removeMin() {
        int min = minHeap.deleteRoot();
        maxHeap.remove(min);
        return min;
    }

    void printHeaps() {
        std::cout << "MaxHeap: ";
        maxHeap.printHeap();
        std::cout << "MinHeap: ";
        minHeap.printHeap();
    }
};
```

Testing the DEPQ

Finally, to test our code, add the following `main` method to the end of the `DoubleEndedPriorityQueue` class.

```
int main() {
    DoubleEndedPriorityQueue depq(10);
    depq.add(30);
    depq.add(20);
    depq.add(50);
    depq.add(15);
    depq.add(10);

    std::cout << "Added elements: 30, 20, 50, 15, 10" <<
        std::endl;
    std::cout << "Max Element Removed: " << depq.removeMax() <<
        std::endl;
    std::cout << "Min Element Removed: " << depq.removeMin() <<
        std::endl;

    std::cout << "Remaining elements in MaxHeap:" << std::endl;
    depq.printHeaps();

    return 0;
}
```

Complete Code File

Here is the complete code for `DoubleEndedPriorityQueue.cpp`:

```
// Base HeapImplementation class
class HeapImplementation {
protected:
    int* heap;
    int size;
    int capacity;

public:
    HeapImplementation(int capacity) : size(0),
        capacity(capacity) {
        heap = new int[capacity];
    }

protected:
    void resize() {
        capacity *= 2;
        int* resizedHeap = new int[capacity];
        for (int i = 0; i < size; i++) {
            resizedHeap[i] = heap[i];
        }
        delete[] heap;
        heap = resizedHeap;
    }

public:
    void insert(int value) {
        if (size == capacity) {
            resize();
        }
```

```
        heap[size] = value;
        heapifyUp();
        size++;
    }

    virtual void heapifyUp() {
        int index = size;
        while (hasParent(index) && parent(index) < heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

public:
    int deleteRoot() {
        if (size == 0) {
            throw std::out_of_range("Heap is empty");
        }
        int root = heap[0];
        heap[0] = heap[size - 1];
        size--;
        heapifyDown();
        return root;
    }

    virtual void heapifyDown() {
        int index = 0;
        while (hasLeftChild(index)) {
            int largerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) >
        leftChild(index)) {
                largerChildIndex = getRightChildIndex(index);
            }
            if (heap[index] < heap[largerChildIndex]) {
                swap(index, largerChildIndex);
            } else {
                break;
            }
            index = largerChildIndex;
        }
    }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    bool hasParent(int i) { return getParentIndex(i) >= 0; }
    int parent(int i) { return heap[getParentIndex(i)]; }
    int getParentIndex(int i) { return (i - 1) / 2; }
    bool hasLeftChild(int i) { return getLeftChildIndex(i) <
        size; }
    bool hasRightChild(int i) { return getRightChildIndex(i) <
        size; }
    int leftChild(int i) { return heap[getLeftChildIndex(i)]; }
    int rightChild(int i) { return heap[getRightChildIndex(i)];
        }
```

```
    int getLeftChildIndex(int i) { return 2 * i + 1; }
    int getRightChildIndex(int i) { return 2 * i + 2; }

public:
    bool remove(int element) {
        for (int i = 0; i < size; i++) {
            if (heap[i] == element) {
                swap(i, size - 1);
                size--;
                heapifyDown();
                return true;
            }
        }
        return false;
    }

    void printHeap() {
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
};

// MaxHeap class
class MaxHeap : public HeapImplementation {
public:
    MaxHeap(int capacity) : HeapImplementation(capacity) {}
};

// MinHeap class
class MinHeap : public HeapImplementation {
public:
    MinHeap(int capacity) : HeapImplementation(capacity) {}

    void heapifyUp() override {
        int index = size;
        while (hasParent(index) && parent(index) > heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

    void heapifyDown() override {
        int index = 0;
        while (hasLeftChild(index)) {
            int smallerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) <
        leftChild(index)) {
                smallerChildIndex = getRightChildIndex(index);
            }
            if (heap[index] > heap[smallerChildIndex]) {
                swap(index, smallerChildIndex);
            } else {
                break;
            }
            index = smallerChildIndex;
```

```
        }
    }
};

// DoubleEndedPriorityQueue class
class DoubleEndedPriorityQueue {
private:
    MaxHeap maxHeap;
    MinHeap minHeap;

public:
    DoubleEndedPriorityQueue(int capacity) : maxHeap(capacity),
        minHeap(capacity) {}

    void add(int element) {
        maxHeap.insert(element);
        minHeap.insert(element);
    }

    int removeMax() {
        int max = maxHeap.deleteRoot();
        minHeap.remove(max);
        return max;
    }

    int removeMin() {
        int min = minHeap.deleteRoot();
        maxHeap.remove(min);
        return min;
    }

    void printHeaps() {
        std::cout << "MaxHeap: ";
        maxHeap.printHeap();
        std::cout << "MinHeap: ";
        minHeap.printHeap();
    }
};

int main() {
    DoubleEndedPriorityQueue depq(10);
    depq.add(30);
    depq.add(20);
    depq.add(50);
    depq.add(15);
    depq.add(10);

    std::cout << "Added elements: 30, 20, 50, 15, 10" <<
        std::endl;
    std::cout << "Max Element Removed: " << depq.removeMax() <<
        std::endl;
    std::cout << "Min Element Removed: " << depq.removeMin() <<
        std::endl;

    std::cout << "Remaining elements in MaxHeap:" << std::endl;
    depq.printHeaps();

    return 0;
```

```
}
```

Formative Assessment 1

Formative Assessment 2